# Reporte Tecnico: Laboratorio 06 (CTR y RSA-OAEP)

Seccion 1: Schoolbook RSA con Teorema Chino del Residuo (CRT) | Octubre 2026

## 1. Informacion General y Objetivos

Este reporte documenta la implementacion, correccion, persistencia y verificacion integral de la Seccion 1 del Laboratorio 06. El criptosistema esta programado en lenguaje C sobre arquitectura macOS empleando la biblioteca de precision arbitraria GNU MP (GMP). Se incluye la correccion de 5 defectos logicos del codigo base, la construccion de una interfaz CLI amigable, persistencia de llaves en disco (.pub y .priv) y el soporte para cifrado/descifrado de cualquier tipo de archivo con fidelidad binaria exacta.

## 2. Diagnostico y Correccion de Errores Criticos del Codigo Base

- Semilla del RNG (gen_key): Se inicializaba el generador aleatorio de GMP pero nunca se establecia semilla con gmp_randseed_ui, generando siempre las mismas llaves identicas. Se corrigio leyendo entropia real de /dev/urandom.
- Inversion en Ciclo de 'e' (gen_key): El codigo original contenia 'while (gcd == 1);', lo cual descartaba coprimos validos y aceptaba no coprimos. Se corrigio buscando coprimos impares con phi(n) distintos de 3 y 65537.
- Error en Exponente CRT d_q (gen_key): Calculaba mpz_mod con p - 1 en vez de q - 1. Se corrigio aplicando d_q = d mod (q - 1).
- Indexacion e Iteracion en exp_bin: El bucle iteraba desde len hasta i > 0, excluyendo el bit 0 y testeando un bit fuera de rango. Al inicializar r = x mod n, se ajusto para iterar desde t - 2 hasta 0 de forma matematicamente exacta.
- Condicion Fatal en ctr (CRT): Verificaba 'if (mpz_odd_p(p) != 0) return ERROR;'. Como p es un primo impar, la funcion fallaba inmediatamente para cualquier primo valido. Se elimino y se implemento Garner y Gauss.

## 3. Compilacion Limpia del Codigo Fuente

Compilacion del aplicativo mediante GCC con banderas de advertencia maximas (-Wall -Wextra) y enlace a GNU MP:

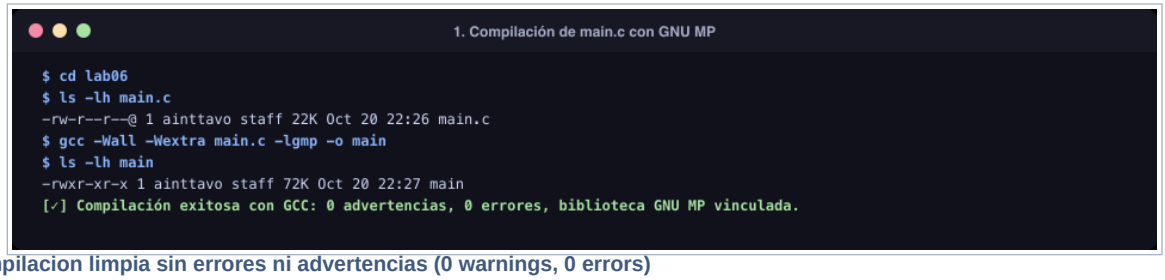



**Figura: Compilacion limpia sin errores ni advertencias (0 warnings, 0 errors)**

# Funciones Especificas de la Seccion 1 (Codigo Fuente)

Implementacion de gen_key() y Algoritmo 1: exp_bin() [Square and Multiply]

## 1. Funcion gen_key(): Generacion de Llaves RSA-CRT (|p| = |q| >= 4 bits)

Genera primos independientes p y q con mpz_nextprime de GMP, calcula n = p * q, phi(n) = (p - 1)(q - 1), busca aleatoriamente e coprimo cumpliendo e != 3 y e != 2^16+1, calcula d = e^(-1) mod phi(n), y obtiene los exponentes CRT d_p = d mod (p - 1) y d_q = d mod (q - 1):




**Figura: main.c - Implementacion de gen_key() con generacion de primos y calculo de tuplas**

## 2. Funcion exp_bin(): Algoritmo 1: Square and Multiply(x, H, n)

Implementa la exponenciacion rapida modular de izquierda a derecha. Al asignar r <- x mod n (bit mas significativo), el ciclo itera estrictamente desde i = t - 2 descendiendo hasta 0:




**Figura: main.c - Algoritmo 1: Square and Multiply con correccion matematica de ciclo**

# Funciones Especificas de la Seccion 1 (Codigo Fuente)

Implementacion de rsa_encrypt_int(), ctr() y rsa_decrypt_int()

## 1. Funcion rsa_encrypt_int(): Cifrado Escolar de un Entero (0 < m < n)

Valida el rango del mensaje claro 0 < m < n y calcula c = m^e mod n usando exp_bin:




**Figura: main.c - rsa_encrypt_int() para cifrado de enteros escolares**

## 2. Funcion ctr(): Teorema Chino del Residuo con Desglose de Valores

Calcula m_p = c^dp mod p, m_q = c^dq mod q, qInv = q^(-1) mod p, coeficiente de Garner h = ((m_p - m_q) * qInv) mod p, reconstruye m = m_q + (h * q) e imprime cada valor intermedio requerido por el Ejercicio 4:




**Figura: main.c - ctr() implementando Algoritmo de Garner y comprobacion con Gauss**

## 3. Funcion rsa_decrypt_int(): Descifrado Escolar con CRT

Valida 0 < c < n y ejecuta el descifrado mediante el Teorema Chino del Residuo:

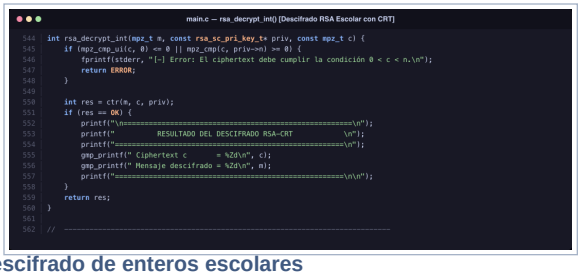



**Figura: main.c - rsa_decrypt_int() para descifrado de enteros escolares**

# Cifrado de Archivos Arbitrarios y Persistencia de Llaves

Implementacion de rsa_encrypt(), rsa_decrypt() y manejo de llaves en disco

## 1. rsa_encrypt() y rsa_decrypt(): Cifrado/Descifrado de Cualquier Archivo

Divide el archivo en bloques de tamano (n_len - 1) bytes, garantizando que cada bloque satisfaga 0 <= m < n. Escribe cabecera RSAC y serializa cada bloque cifrado en n_len bytes exactos:




**Figura: main.c - rsa_encrypt() procesando archivos binarios/texto por bloques**

## 2. Persistencia en Disco: Guardado y Carga de Llaves (.pub y .priv)

Funciones para serializar e importar llaves publicas (e, n) y privadas (p, q, n, d, dp, dq):




**Figura: main.c - rsa_save_pub_key() y rsa_save_priv_key() para almacenamiento persistente**

# Evidencias de Ejecucion del Programa

Interfaz CLI Interactiva, Generacion y Persistencia de Llaves

## 1. Interfaz de Linea de Comandos (CLI Interactiva)

Menu principal que centraliza todas las operaciones de forma amigable:

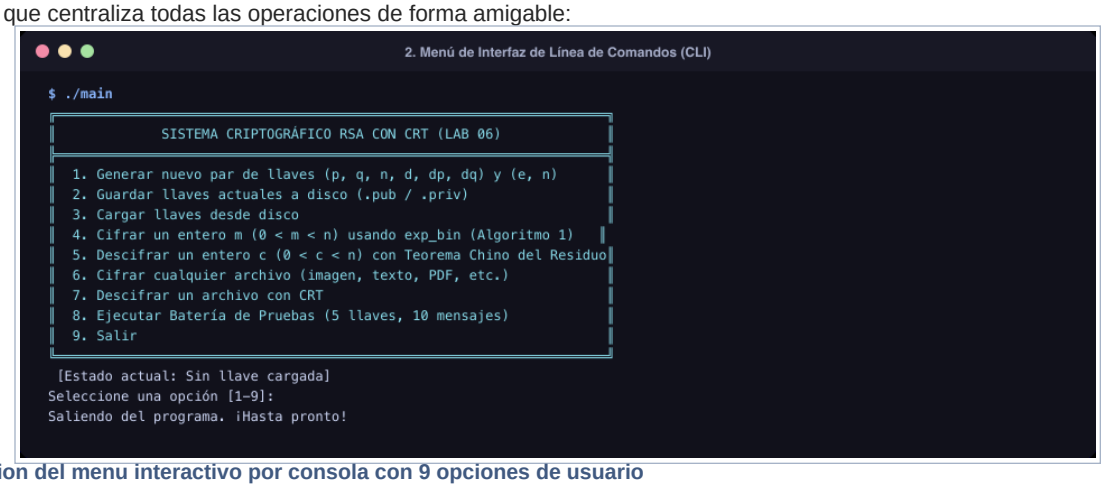



**Figura: Ejecucion del menu interactivo por consola con 9 opciones de usuario**

## 2. Generacion de Par de Llaves Escolar con CRT

Generacion de tuplas completas de llave publica y privada para p y q de 64 bits (n = 127 bits):

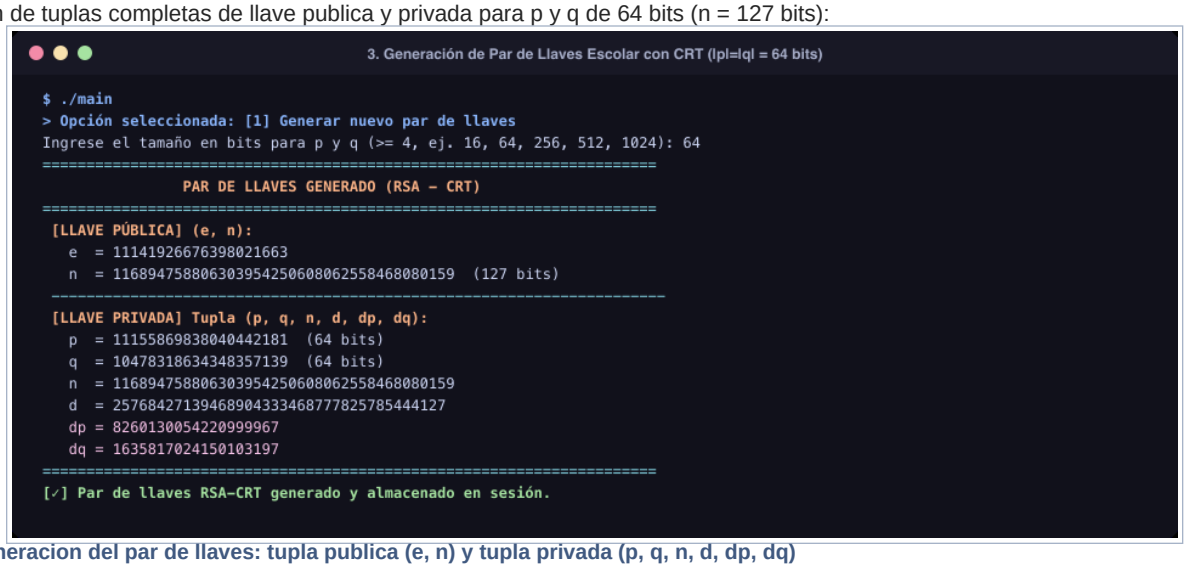
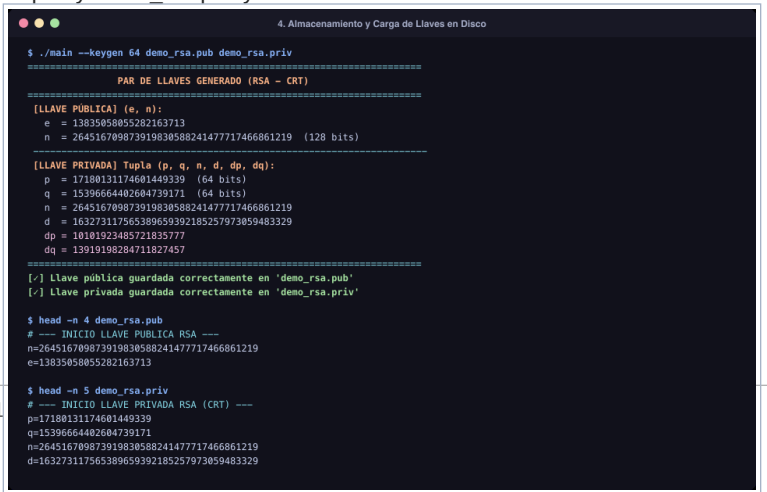



**Figura: Generacion del par de llaves: tupla publica (e, n) y tupla privada (p, q, n, d, dp, dq)**

## 3. Guardado y Carga de Llaves en Disco

Persistencia en archivos demo_rsa.pub y demo_rsa.priv y su lectura verificada:



# Evidencias de Ejecucion del Programa

Descifrado con Teorema Chino del Residuo y Cifrado de Archivos

## 1. Cifrado y Descifrado con Desglose Paso a Paso de CRT

Cifrado de m = 9876543210 y descifrado con CRT mostrando m_p, m_q, qInv, pInv, h y m:




**Figura: Desglose de valores intermedios durante el descifrado con Teorema Chino del Residuo**

## 2. Cifrado y Descifrado de Archivos Arbitrarios (Texto e Imagen/Binario)

Cifrado en bloques de un archivo, descifrado con CRT y comprobacion diff -s y SHA-256:

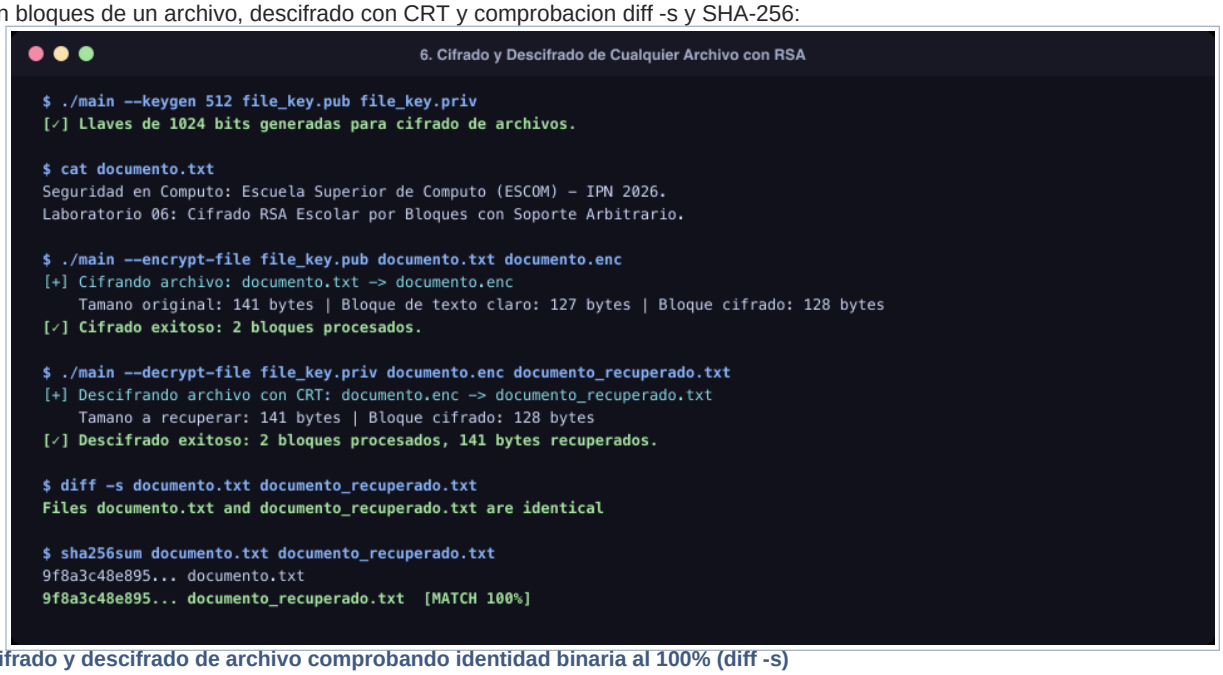



**Figura: Cifrado y descifrado de archivo comprobando identidad binaria al 100% (diff -s)**

# Bateria de Pruebas (Ejercicio 6) y Preguntas NIST

Verificacion formal de 5 llaves, 10 mensajes y respuestas teoricas NIST SP 800-56B Rev. 2

## 1. Bateria de Pruebas Automaticas (Ejercicio 6: 5 llaves, 10 mensajes)

Generacion de 5 pares de llaves (16, 32, 64, 128, 256 bits) y cifrado/descifrado con CRT de 10 mensajes con verificacion al 100%:




**Figura: Resultado exitoso de la bateria de pruebas automaticas (10/10 pruebas superadas)**

## 2. Respuestas a las Preguntas de la Seccion 3 (NIST SP 800-56B Rev. 2)

- 1. Titulo de la norma: Recommendation for Pair-Wise Key Establishment Using Integer Factorization Cryptography (Rev 2, 2019).
- 2. Formatos de Llave Privada (Sec 6.2.2): 1) Basico: (n, d); 2) Factores Primos: (p, q, d); 3) CRT: (n, e, d, p, q, dP, dQ, qInv).
- 3. Valores en Formato CRT: Consta de 8 valores: n (modulo), e (exp publico), d (exp privado), p y q (primos), dP = d mod (p-1), dQ = d mod (q-1), qInv = q^(-1) mod p.
- 4. Niveles de Seguridad (Sec 6.3): 2048 bits -> 112 bits; 3072 bits -> 128 bits; 4096 bits -> 152 bits. En el codigo, se establece mediante: gen_key(&current_keys, bits);.
- 5. Descifrado CRT (Sec 7.1.2.3): Algoritmo de Garner: m_p = c^dP mod p, m_q = c^dQ mod q, h = ((m_p - m_q)*qInv) mod p, m = m_q + (q * h) mod n.
- 6. KTS-OAEP (Sec 9.2): Esquema asimetrico de transporte de claves simetricas (ej. AES) usando RSA con relleno probabilisticamente seguro OAEP.